ASSESSMENT – 1

DESIGN AND ANALYSIS OF ALGORITHMS

Scalability Review of an Online Retail Platform

A analysis of algorithm growth, recurrence relations, search methods, and processing limits

Student Name	Nabilah Thahseen.B
Register Number	1925260057
Course Code / Name	CSA0610 – Design and Analysis of Algorithms
Assessment	Assessment – 1

1. Introduction

An online retail service may begin with a manageable product list but later grow to hundreds of thousands or millions of entries. An algorithm that performs well on a small dataset can become a bottleneck when the amount of data increases. This report studies the growth of several common processing modules and compares their estimated work.

The platform is evaluated at two catalogue sizes: 100,000 items and 1,000,000 items. The modules include duplicate checking, request auditing, two search approaches, a recommendation process, a sales-analysis process, and a report-generation formula.

Assumptions used

• n represents the number of records processed by a module.

• Logarithms are base 2 unless stated otherwise.

• The hypothetical server completes 10 basic operations per second.

• The nightly processing allowance is 600 seconds (10 minutes).

• Therefore, the operation budget is 10 × 600 = 6,000 operations.

These are theoretical estimates based on the stated assumptions; actual execution time depends on hardware, implementation details, and data layout.

2. Task (a) – Identify the Basic Operations

Module / method	Input being processed	Operation counted	Expected growth
M1: Duplicate check	Product records	Compare two product keys	Theta(n squared)
M2: Request audit	Incoming requests	Execute one audit action	Theta(n)
S1: Sequential search	Unsorted product list	Compare key with current item	Theta(n)
S2: Binary search	Sorted index	Compare key with middle item	Theta(log n)
M4: Recommendation (current)	Customer/product data	Combine four half-size recursive results	Theta(n squared)
M4: Recommendation (redesigned)	Customer/product data	Combine two half-size recursive results	Theta(n log n)
M5: Sales analysis	Sales records	Process n log n work per recursive level	Theta(n log squared n)
Report generation	Sales records	Evaluate dominant quadratic work	Theta(n squared)

The growth class describes how work changes as n becomes large. It does not, by itself, give an exact runtime; constants and implementation overhead also matter.

3. Task (b) – Iterative Module Analysis

M1: Duplicate-checking routine

Each product is compared with every product appearing after it. For n items, the inner loop performs n–1 comparisons for the first item, n–2 for the next, and continues until one comparison remains.

$$C(n) = (n - 1) + (n - 2) + \ldots + 2 + 1$$

$$C(n) = n(n - 1) / 2 = (n \text{ squared} - n) / 2$$

The loops do not stop early when a duplicate is found, so the count is independent of the particular product values.

n	Pair comparisons	Interpretation
1,000	499,500	Manageable for a small batch
100,000	4,999,950,000	Very large workload
1,000,000	499,999,500,000	Not practical for the stated limit

Best, average, and worst cases are all Theta(n squared) for this implementation. Auxiliary space is Theta(1), because only a fixed amount of extra storage is required.

M2: Request-audit routine

The outer-loop variable doubles on each pass: 1, 2, 4, 8, and so on. The audit action runs i times during a pass where the current value is i. Thus, the total is a geometric sum.

$$A(n) = 1 + 2 + 4 + \ldots + 2^{(k-1)} = 2^k - 1$$

With k = ceil(log base 2 of n), the operation count is 2^ceil(log base 2 of n) – 1 for the loop condition described in the reference. This is bounded by a constant multiple of n, so the time complexity is Theta(n), not Theta(n log n).

n	Audit operations	Outer-loop passes
1,024	1,023	10
100,000	131,071	17
1,000,000	1,048,575	20

Even though M2 is linear, its operation count is still above 6,000 at one million records.

4. Task (c) – Solve the Binary-Search Recurrence

Binary search cuts the remaining search interval approximately in half after each comparison. Its recurrence can be written as:

T(n) = T(n/2) + 1, with T(1) = 1

Backward substitution

T(n) = T(n/2) + 1

= T(n/4) + 2

= T(n/8) + 3

= T(n/2^i) + i

The base case is reached when n/2^i = 1. Rearranging gives i = log base 2 of n. Therefore, for a power-of-two input:

T(n) = 1 + log base 2 of n

For general positive n, a common worst-case comparison count is floor(log base 2 of n) + 1. At n = 1,000,000, this is 20 comparisons.

Search implementation	Time complexity	Extra space
Iterative binary search	Theta(log n)	Theta(1)
Recursive binary search	Theta(log n)	Theta(log n)

Binary search is highly scalable, but the data must be sorted or stored in a maintained sorted index. Under the hypothetical rate of 10 operations per second, 20 comparisons correspond to about 2 seconds.

5. Task (d) – Recurrence Analysis

M4: Recommendation engine

The original version makes four recursive calls on inputs of half the current size, then performs linear combining work:

T(n) = 4T(n/2) + n

Here a = 4, b = 2 and n^(log base 2 of 4) = n squared. Since the additional n work is smaller than n squared, the Master Theorem gives:

T(n) = Theta(n squared)

A proposed version makes two half-size calls instead:

T(n) = 2T(n/2) + n

Now n^(log base 2 of 2) = n, which matches the combining work. Therefore:

T(n) = Theta(n log n)

Design	Recurrence	Asymptotic result
Current	4T(n/2) + n	Theta(n squared)
Proposed	2T(n/2) + n	Theta(n log n)

The redesign has a much better growth rate. However, improved complexity does not automatically guarantee that a strict operation budget will be met.

M5: Sales-analysis engine

The sales-analysis recurrence is:

T(n) = 2T(n/2) + n log base 2 of n

In the Master Theorem form, a = 2 and b = 2, so n^(log base 2 of 2) = n. The extra work is n log n, which is larger than n by a logarithmic factor. The extended Master Theorem gives:

T(n) = Theta(n log squared n)

Recursion-tree check

Let L = log base 2 of n. At level i there are 2^i subproblems, each of size n/2^i. The combined work at that level is n(L − i). Summing the internal levels and including leaf work produces:

T(n) = n × L(L + 1)/2 + n

For n = 1,024, L = 10, giving 1,024 × 10 × 11 / 2 + 1,024 = 57,344 operations. This is much larger than the 6,000-operation allowance.

6. Task (e) – Best, Average, and Worst Search Cases

S1: Sequential search

Sequential search checks entries in order. If the target is first, it takes one comparison; if it is last or absent, it may inspect the whole list.

Best: 1 = Theta(1) Worst: n = Theta(n)

If a successful target is equally likely to be at any position, the average number of comparisons for successful searches is:

Cavg(n) = (1 + 2 + … + n)/n = (n + 1)/2

If some searches fail, the overall average increases because an unsuccessful search normally checks all n entries.

S2: Binary search

Binary search uses the middle item to discard roughly half the remaining range. Its best case is one comparison; its worst-case count is floor(log base 2 of n) + 1.

Best: 1 = Theta(1) Worst: floor(log base 2 of n) + 1 = Theta(log n)

Case at n = 1,000,000	Sequential search	Binary search
Best	1 comparison	1 comparison
Average (illustrative)	About 750,000 if half succeed and half fail	About 19–20 comparisons
Worst	1,000,000 comparisons	20 comparisons

The average figures for binary search are estimates, since the exact average depends on the tree shape and distribution of successful and unsuccessful searches. In a large catalogue, binary search is generally preferable when a sorted index is available.

7. Task (f) – Prove the Report Generator Bound

The report generator is represented by the function:

f(n) = 3n squared + 10n log base 2 of n + 50

To prove Theta(n squared), show that positive constants c1 and c2 exist such that c1 n squared ≤ f(n) ≤ c2 n squared for all n beyond a threshold.

Lower bound

For n ≥ 1, the terms 10n log base 2 of n and 50 are non-negative. Therefore:

f(n) ≥ 3n squared

This establishes Omega(n squared), with c1 = 3.

Upper bound

For n ≥ 1, log base 2 of n ≤ n and 50 ≤ 50n squared. Hence:

10n log base 2 of n ≤ 10n squared

f(n) ≤ 3n squared + 10n squared + 50n squared = 63n squared

This establishes O(n squared), with c2 = 63. Taking c1 = 3, c2 = 63, and n0 = 1 gives:

3n squared ≤ f(n) ≤ 63n squared

Therefore, f(n) = Theta(n squared). It cannot be O(n), because its quadratic term alone grows faster than any fixed constant multiplied by n.

8. Task (g) – Processing-Window Decision

At 10 operations per second, a 600-second window allows at most 6,000 basic operations. Estimated time is calculated as operation count divided by 10.

Module	Operations at n = 1,000,000	Estimated time	Within 10 min?
M1 – Duplicate check	499,999,500,000.00	49,999,950,000.00 s	No
M2 – Request audit	1,048,575.00	104,857.50 s	No
S2 – Binary search	20.00	2.00 s	Yes
M4 – Current design	1,999,999,000,000.00	199,999,900,000.00 s	No
M4 – Proposed design	20,931,568.57	2,093,156.86 s	No
M5 – Sales analysis	209,599,497.10	20,959,949.71 s	No
Report generator	3,000,199,315,735.69	300,019,931,573.57 s	No

Decision: only binary search stays within the stated limit. All other counts exceed 6,000 operations at one million records. These values are theoretical and should not be interpreted as real server benchmarks.

9. Recommendations

• Replace the all-pairs duplicate check with a hash set keyed by a stable product identifier or by the required combination of product fields. Expected time can improve to Theta(n), with additional memory use.

• Keep a sorted search index updated as catalogue records change so that product lookups can use binary search.

• Use the redesigned recommendation recurrence because Theta(n log n) scales better than Theta(n squared). Profile it after implementation.

• For sales analytics, investigate whether repeated sorting or repeated processing can be replaced with a single sort followed by a merge or grouped aggregation.

• Inspect the report generator for nested scans and repeated work. Hash-based lookups or one-pass aggregation may remove unnecessary quadratic processing.

• For the rate limiter, process audit records in batches or distribute work only if correctness and security rules remain intact.

10. Conclusion

This review shows why asymptotic analysis is important before a retail platform expands. Quadratic algorithms become costly as the catalogue grows, while logarithmic search requires very few comparisons. The recommendation redesign improves the growth rate, and the sales-analysis recurrence grows as Theta(n log squared n). Under the assignment's assumed server rate and time window, binary search is the only evaluated module that meets the 10-minute requirement at one million records. The other modules need optimisation and fresh operation-count checks.

Appendix – Pseudocode for an Improved Duplicate Check

```
seen = empty set
for each product in catalogue:
    key = (product.title, product.seller)
    if key is in seen:
        mark product as a possible duplicate
    else:
```

```
                add key to seen
```

Expected time: Theta(n) with an appropriate hash set. Extra space: Theta(n). Hashing is an optimisation option; the chosen key must match the platform's definition of a duplicate.

Academic Integrity Statement

I, Nabilah Thahseen.B (Register Number: 1925260057), declare that this submission has been prepared for this assessment and that I will follow the academic integrity requirements of my institution.

Student Signature: Nabilah.B

Date: 12-10-2026